

```

#include<iostream>
#include<cstdlib>
#include<ctime>

using namespace std;

#define SIZE 10
#define UP 0
#define DOWN 1
#define LEFT 2
#define RIGHT 3

class Node {
public:
    Node() {
        Node(0, 0);
    }

    Node(int r, int c) {
        row = r;
        col = c;
    }

    int getRow() { return row; }
    int getCol() { return col; }

    void setRow(int r) {
        if (r >= 0 && r < SIZE)
            row = r;
    }

    void setCol(int c) {
        if (c >= 0 && c < SIZE)
            col = c;
    }

private:
    int col, row;
};

class List {
public:
    List() {
        top = 0;
    }

    // Add an element (row and column) to the list
    void addElement(int r, int c) {
        if (top < SIZE * SIZE) { // Check not to exceed the list's size
            data[top][0] = r;    // Row
            data[top][1] = c;    // Column
            top++;
        }
    }

    // Remove the last element from the list
    void removeElement() {
        if (top > 0) {
            top--;
        }
    }

```

```
}
```

```
// Print the directions (path) based on grid movements
```

```
void printPath() {
```

```
    for (int i = 1; i < top; i++) {
```

```
        if (data[i][0] == data[i - 1][0] - 1 && data[i][1] == data[i - 1][1])
```

```
            cout << "UP\n";
```

```
        else if (data[i][0] == data[i - 1][0] + 1 && data[i][1] == data[i - 1][1])
```

```
            cout << "DOWN\n";
```

```
        else if (data[i][1] == data[i - 1][1] - 1 && data[i][0] == data[i - 1][0])
```

```
            cout << "LEFT\n";
```

```
        else if (data[i][1] == data[i - 1][1] + 1 && data[i][0] == data[i - 1][0])
```

```
            cout << "RIGHT\n";
```

```
    }
```

```
}
```

```
private:
```

```
    int data[SIZE * SIZE][2]; // Array to store the row and column of the path
```

```
    int top;
```

```
};
```

```
class Maze {
```

```
public:
```

```
    Maze() {
```

```
        initMaze(SIZE);
```

```
    }
```

```
// Initialize the maze with walls and open spaces
```

```
void initMaze(int s) {
```

```
    maze = new int*[s];
```

```
    for (int i = 0; i < s; i++) {
```

```
        maze[i] = new int[s];
```

```
    }
```

```
    srand(time(0)); // Seed the random number generator
```

```
// Initialize maze grid with open spaces (0)
```

```
    for (int i = 0; i < s; i++) {
```

```
        for (int j = 0; j < s; j++) {
```

```
            maze[i][j] = 0;
```

```
        }
```

```
    }
```

```
// Randomly generate walls (1), covering 20% of the maze
```

```
int wallCount = (s * s) * 0.2;
```

```
while (wallCount > 0) {
```

```
    int r = rand() % s;
```

```
    int c = rand() % s;
```

```
// Ensure start [0,0] and end [s-1, s-1] points are not walls
```

```
if ((r != 0 || c != 0) && (r != s - 1 || c != s - 1) && maze[r][c] == 0) {
```

```
    maze[r][c] = 1; // Place a wall
```

```
    wallCount--;
```

```
}
```

```
}
```

```
}
```

```
// Find and return the path from start [0,0] to end [SIZE-1, SIZE-1]
```

```
List* getPath() {
```

```
    List* path = new List();
```

```

    bool visited[SIZE][SIZE] = { false }; // Track visited cells
    findPath(0, 0, visited, path); // Start pathfinding from [0,0]
    return path;
}

// Display the maze structure
void printMaze() {
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            if (maze[i][j] == 0)
                cout << " ";
            else if (maze[i][j] == 1)
                cout << "**";
        }
        cout << endl;
    }
}

private:
    int** maze;

    // Recursively explore the maze for a path
    bool findPath(int r, int c, bool visited[SIZE][SIZE], List* path) {
        // Return false if out of bounds, on a wall, or already visited
        if (r < 0 || c < 0 || r >= SIZE || c >= SIZE || maze[r][c] == 1 || visited[r][c])
            return false;

        // Mark the cell as visited
        visited[r][c] = true;

        // Add current position to the path
        path->addElement(r, c);

        // If we've reached the end, return true
        if (r == SIZE - 1 && c == SIZE - 1)
            return true;

        // Explore neighboring cells (right, down, left, up)
        if (findPath(r, c + 1, visited, path) ||
            findPath(r + 1, c, visited, path) ||
            findPath(r, c - 1, visited, path) ||
            findPath(r - 1, c, visited, path)) {
            return true;
        }

        // If no path is found, backtrack
        path->removeElement();
        return false;
    }
};

int main() {
    Maze* maze = new Maze();
    maze->printMaze();
    maze->getPath()->printPath();
}

```